Syrudas AI: A Local-First AI Workspace with Pluggable Model Providers

Version 1.0.0 · July 2026 · Len · MIT License

In brief

Syrudas AI is a program you install on your own Windows PC. It gives you a chat window for talking to an AI, much like the well-known online assistants, with one difference: nothing you type has to leave your computer.

You choose which AI answers you. It can be one running entirely on your own machine — downloaded once and stored locally — so your conversations never touch the internet at all. Or it can be one of the commercial services, if you have an account and prefer their answers. Either way you use the same window, the same saved history, and the same settings. Switching between them takes a few seconds and changes nothing else about how the program behaves.

Beyond conversation, the AI can be given permission to *do* things: run commands, read and write files in folders you nominate, search the web, and remember facts between conversations. Anything genuinely risky stops and asks you first — every time, showing you exactly what it proposes to do before it does it. It can also read your own documents, such as a folder of notes or reports, and answer questions about them without uploading anything.

The rest of this paper explains how the program is built and why. It is written for readers who want that level of detail. **Key terms**, immediately below, defines the technical vocabulary that recurs throughout; Sections 1 and 2 give the reasoning and the overall shape; the remaining sections take each part in turn, and Section 19 is a candid account of what the system does not yet do well.

Contents

Key terms

Term	What it means here
Model	The AI itself — the component that reads your message and writes a reply.
Provider	A service that runs a model. May be software on your own PC or a company's website.
Local model	A model running on your own hardware, with no internet involved.
Token	The unit models read and write in — roughly three-quarters of a word.
Context window	How much of a conversation a model can consider at once, measured in tokens. Older material falls out when it fills.
Streaming	Showing a reply word by word as it is produced, rather than waiting for all of it.
Agent mode	A setting where the AI may take actions on your behalf, not just reply.
Tool	One specific action the AI can take, such as reading a file or searching the web.
Tool call	The AI's request to use a tool, which the program then carries out.
Approval gate	The prompt asking your permission before a risky tool runs. Nothing risky bypasses it.
Plugin	A small file that teaches the program to talk to a new provider, added without changing the program itself.
MCP	Model Context Protocol — a shared standard letting outside programs offer tools to the AI.
Retrieval	Searching your own documents for passages relevant to a question, and showing them to the model.
Embedding	A numeric fingerprint of a passage of text, used to find passages with similar meaning.
Prompt injection	Text hidden in a web page or document that tries to issue instructions to the AI as though you had written them.
Loopback	The address `127.0.0.1`, meaning "this computer only" — unreachable from the network.

Abstract

Syrudas AI is a self-hosted AI workspace for Windows that runs entirely on the user's own machine. It offers streaming chat, an autonomous agent mode with tool use and Model Context Protocol (MCP) support, durable cross-conversation memory, local retrieval over the user's own files, a deep-research pipeline, a blind model-comparison arena, an AI-assisted writing editor, a hardware-aware model cookbook, and an OpenAI-compatible endpoint that lets other tools use the same configured backends. What makes this breadth tractable for a single maintainer is a deliberate architectural bet: **the model is a plugin.** Every backend — local weights or a hosted frontier API — reaches the application through one small contract, and nothing above that contract knows which model is answering. This paper is a from-first-principles account of the system: the thesis, the abstraction that carries it, the normalized data model that makes the abstraction cheap, how each feature falls out of that model as a consequence rather than a bolt-on, and the security posture that a local, single-user tool can honestly promise.

---

1. Motivation

In plain terms. Why build this at all. Most AI chat happens on somebody else's computer: you type, your words travel to a company's servers, and your conversations stay there. This project starts from the opposite position — the program runs on your machine — and adds one idea. The AI itself is the part of this technology that changes fastest, so it is built to be the easiest piece to swap out, and everything you actually want to keep is arranged so it does not have to change when the AI does.

Using a large language model increasingly means renting a seat in a vertically integrated stack: one vendor's model, behind one vendor's application, with the user's conversations, keys, and files living on one vendor's servers. That arrangement is convenient and, for many, sufficient. It is also fragile in a particular way: it couples the *interface* you rely on to the *model* you happen to prefer this quarter, and it couples both to a third party's continued goodwill about your data. Projects such as PewDiePie's **Odysseus** (2026) showed that a meaningful audience wants the opposite: a workspace you run yourself, where the conversation history, the API keys, and the documents never leave the machine.

Syrudas AI begins from that local-first premise but pushes on one idea harder than most such projects do. Model backends are the fastest-moving, least-durable part

of this ecosystem — new weights and new APIs arrive monthly, and today's best choice is next quarter's fallback. A workspace built around any specific model, or any specific provider's SDK, inherits that churn. So the organizing principle here is that **the model is the most replaceable component, and the architecture should treat it that way.** Everything the user actually values — their history, their tools, their documents, the agent runtime, the UI — sits above a thin seam, and the model plugs into that seam.

Five concrete goals follow from this stance:

1. **Any model, one workspace.** Local weights via Ollama or LM Studio, and hosted models via OpenRouter or first-party APIs, share one UI, one conversation store, and one agent runtime.
2. **Local-first and private by construction.** The server binds to `127.0.0.1` and rejects non-loopback requests. There is no account, no telemetry, and no cloud component. Keys are stored locally and masked in every response.
3. **Agentic, but consent-gated.** The model can plan and act — run commands, read and write files, search and read the web, remember facts, search indexed documents — with an explicit, per-action human gate on the dangerous parts.
4. **A hub, not a silo.** The workspace speaks the OpenAI wire dialect in both directions, so external tools can use its configured models as a service.
5. **Shippable to a non-developer.** A self-contained portable folder that unzips and double-clicks, with zero-configuration onboarding when a local model server is already present.

The rest of this paper traces how a single abstraction, enforced everywhere, lets those goals coexist inside a codebase one person can hold in their head.

2. System overview

In plain terms. What the pieces are. There are two halves. One is a small program that runs quietly in the background on your PC and does the actual work: talking to the AI, saving your conversations, running anything the AI is allowed to run. The other is the window you look at. They talk to each other over your own machine and nowhere else. For everyday use both halves are wrapped into one folder with one thing in it to double-click, so you never see the seam.

Syrudas is a two-tier application with a deliberately small surface between the tiers. The **backend** is Python 3.13 on FastAPI: it serves a REST and streaming

API, owns a single SQLite database, runs the agent and research loops, and hosts the provider, MCP, and retrieval subsystems. The **frontend** is a React 19 / TypeScript single-page app built with Vite — chat, editor, arena, cookbook, and settings — compiled to static assets that the backend serves. The same compiled bundle runs unmodified in an ordinary browser tab and in the packaged desktop window; nothing about it assumes which.

The packaged desktop application wraps both tiers in a native window (pywebview over the Windows WebView2 runtime), with the FastAPI server running on a background thread of the same process. There is no second process to supervise and no port coordination beyond a single fixed local port; closing the window stops the server, and launching the executable again while it is running simply opens a new window onto the existing instance.

State is intentionally boring, which is a feature rather than an apology. One SQLite file holds conversations, messages, provider instances, MCP registrations, memories, indexed-document chunks and their embeddings, editor documents, arena results, and settings. One workspace folder holds files the agent writes. In the portable build both live directly beside the executable, so copying the folder copies the entire installation and deleting it removes every trace — no registry keys, no scattered application-data directories. Lightweight session state — the open view and conversation, plus the model and theme choices — is kept in the frontend's local storage (persisted across desktop restarts by the WebView2 storage path), so the app reopens exactly where the user left off.

3. The provider contract

In plain terms. A *provider* is whatever is actually running the AI — a program on your own PC, or a company's service you have an account with. This section sets out the short list of things any provider must be able to do to be usable here: say which models it offers, and hold a conversation. That list is the whole agreement. Anything that can meet it can be plugged in, and the rest of the program never needs to know which one answered — which is what makes adding a new AI service a small, contained job instead of a rewrite.

Everything in the previous section rests on a single class. A provider *type* is a Python class; a provider *instance* is that class configured with user data — a base URL, an API key — in the settings UI and persisted in SQLite. The entire contract is five methods, three of them optional:

```
class ModelProvider(ABC):
    type_id: str                        # "openai_compat", "anthropic", ...
    display_name: str
    config_fields: list[ConfigField]    # declares the settings form to render

    async def list_models(self) -> list[ModelInfo]: ...
    async def chat(self, model, messages, tools=None, params=None)
        -> AsyncIterator[StreamEvent]: ...
    async def context_tokens(self, model) -> int | None: ...        # optional
    async def embed(self, model, texts) -> list[list[float]]: ...   # optional
    async def check(self) -> str: ...                                # optional
```

Three decisions turn this small surface into the load-bearing wall of the whole system.

A normalized interior. Messages, tool specifications, tool calls, and stream events are defined exactly once, in an OpenAI-flavored shape (Section 4). A provider adapter's only job is to translate between that shape and its backend's wire format, at the edge. The chat pipeline, the agent loop, the persistence layer, and the UI are all written against the normalized types and never learn which backend produced them. Adding support for a new API is therefore a self-contained translation file, not a change that ripples through the application.

Streaming as the only mode. `chat()` returns an async iterator of stream events; there is no separate "get a completion" method. A non-streaming response is simply a stream that ends quickly. This collapses what is usually two code paths — with two sets of bugs — into one, and it means every consumer, from the chat view to the deep-research synthesizer, handles partial output the same way.

Discovery by drop-in. The registry imports its builtin adapters statically, because a PyInstaller bundle is invisible to `pkgutil`'s directory scanning and dynamic-only

imports would never be packaged at all. On top of that, it scans a user-writable `plugins/` folder next to the executable at startup. The consequence is unusual and, for a local tool, delightful: a user can teach the *packaged* application to speak to a brand-new backend by dropping a single Python file beside the exe and restarting — no rebuild, no toolchain.

The single builtin adapter, `openai_compat`, is enough for most of the ecosystem, because Ollama, LM Studio, the llama.cpp server, vLLM, OpenRouter, and OpenAI itself all speak the same dialect. Optional connectors for **Anthropic (Claude)** and **Google (Gemini)** ship as drop-in plugins that translate those non-OpenAI dialects.

The two optional methods beyond `check()` are the places the contract grew a new *capability* rather than a new backend, and both follow the same rule: a provider that cannot do the thing says so, and the caller degrades rather than guesses. Providers implementing `embed()` become eligible to power the retrieval subsystem (Section 8); those that do not raise `NotImplementedError` and are omitted from the embedding-model picker. `context_tokens()` reports the usable context window of a given model, which is what lets history be trimmed to the model actually selected (Section 5) instead of to one number for all of them. Its contract is explicitly that it must never guess — returning `None` means "I cannot find out" and drops the caller to a conservative fixed budget, because a window reported too large makes the backend silently discard the oldest messages, taking the system prompt and the user's request with them. The builtin adapter fills it in from whatever the backend volunteers: OpenRouter's `context_length`, or the architecture-prefixed `*.context_length` key in an Ollama model's metadata.

4. The normalized interior

In plain terms. Different AI services describe the same simple things — "here is a message", "here is a reply" — in different formats. Rather than let those differences leak through the whole program, each one is translated once at the edge into a single shared vocabulary. Inside, a message is just a message, whatever produced it. Only the small translator written for each service knows the difference, which is why the rest of the program stays the same size as more services are added.

The abstraction in Section 3 is only cheap because the types it exchanges are few and stable. They live in one module and are shared verbatim by every provider, route, and persistence call:

- `Message` — a role (`system` / `user` / `assistant` / `tool`), text

content, an optional list of tool calls (on assistant messages), and an optional tool-call id (on tool-result messages).

- `ToolSpec` — a tool offered to the model: name, description, and a

JSON-Schema parameter object.

- `ToolCall` — a model's request to invoke a tool: id, name, parsed

arguments.

- `StreamEvent` — one event in a provider's output stream. The vocabulary

is small and closed: `text_delta`, `tool_call`, `usage`, `error`, `done`.

The event vocabulary is where the "streaming as the only mode" decision pays off structurally. The chat route serializes these events to the client as newline-delimited JSON, but along the way it *interleaves* events the provider never produced — an `approval_required` when the agent needs consent, a `research_status` as the research pipeline works, a `tool_result` after a tool runs. Because the client already consumes one flat event stream, plain chat, agent tool activity, research progress, and consent prompts all arrive over the same channel and are rendered by the same reader. A new interactive behavior is usually a new event type, not a new transport.

Choosing an OpenAI-flavored shape for the interior was pragmatic rather than ideological: most backends already speak it, so most adapters are nearly identity functions, and the `/v1` hub (Section 12) can re-emit the interior almost directly. The cost is that genuinely non-OpenAI providers (Anthropic's content-block model, Gemini's `parts`) do real translation work — but that work is confined to their adapter, which is exactly where the contract says it belongs.

5. The chat pipeline

In plain terms. What happens between pressing Enter and seeing an answer. The reply arrives a piece at a time rather than all at once, which is why it appears to type itself. Two safeguards are worth knowing about: your conversation is written

down as it arrives rather than only at the end, so an interruption does not lose it; and if you edit or delete a conversation while a reply is still on its way, that reply cannot come back and overwrite what you did. The section closes with the one real weakness here — how much of a long conversation the AI is shown, and why that limit is currently cruder than it should be.

`POST /api/chat` accepts a message, persists it, and returns a streamed NDJSON body: one JSON event per line, consumed on the client by a `ReadableStream` reader. NDJSON-over-POST was chosen over two obvious alternatives for concrete reasons. WebSockets bring a connection lifecycle — reconnection, heartbeats, state — that a request/response interaction does not need. Server-Sent Events cannot carry a POST body, which a chat turn requires. A streamed POST response is the smallest thing that works, and it composes cleanly with the interleaved event model of Section 4.

Two robustness properties are worth naming because they recur throughout the system. First, **messages are persisted as they complete**, so a conversation survives an application restart or a version upgrade, and the partial assistant text produced before a *provider* failure mid-stream is still written. A client that disconnects is a different case — the generator is torn down rather than allowed to finish — and Section 6 describes the exit contract that governs it. Second, each conversation carries a **generation counter**. When history is rewritten out of band — a rewind, an edit-and-resend, a delete — the counter is bumped, and any still-running stream checks it before persisting. This is the mechanism that stops a "zombie" stream (client gone, cancellation not yet observed) from orphaning tool messages or resurrecting rows in a conversation the user already deleted. The same guard protects the research pipeline, which also persists into a conversation.

Treating the model as a plugin has a consequence for conversation state that is easy to miss: a workspace whose model is interchangeable will accumulate threads answered by *different* models. A conversation therefore records the provider and model it last used, and reopening one restores that pick. Without it, a global picker silently applies whatever model was last chosen anywhere to a thread every previous turn answered with another — a wrong-model reply that leaves no trace in the transcript. The same persistence instinct applies to the `usage` event: the token counts a provider reports at the end of a turn are stored on the assistant message they belong to, so per-reply accounting survives a reload instead of living only in the open tab. In agent mode, where one turn may make

several provider calls, counts are recorded per step against the message that step produced.

Long conversations are trimmed before a turn begins: the system prompt and the newest messages always survive, the oldest turns fall off first, and a tool result whose originating tool call was trimmed away is dropped rather than sent as a dangling reference that confuses backends.

The budget itself is derived from the model actually selected. `context_tokens()` (Section 3) is asked what the chosen model's window is; the answer is converted at a deliberately pessimistic 3.5 characters per token, with a reserve held back for the reply and for the tool schemas an agent turn sends on every request. A provider that will not say falls back to a fixed conservative count, and the derived budget never drops *below* that floor merely because a window is small. The asymmetry is intentional throughout: sending less than the window could hold costs some context, while sending more makes the backend evict from the front — the system prompt and the user's original question — which is the failure that looks like the model ignoring what it was asked.

In agent mode the trim runs **every step, not once per turn**. This is the correction that matters most in practice, because a tool-heavy run appends results as it goes: trimming once before the first step meant a run could grow past its budget mid-turn and evict the user's request while still working on it. Both of these were listed as open problems in the previous edition of this paper and are now closed; what remains imperfect is the character-per-token approximation itself, which is a heuristic standing in for a real tokenizer.

6. Agent mode

In plain terms. Normally the AI only writes back to you. In agent mode it can also *do* things — run a command, read or write a file, look something up on the web. It works in a loop: decide what to do, do it, look at what came back, decide again, and stop when the job is done or when it hits a fixed ceiling on how many turns it may take. Anything genuinely risky pauses and asks your permission first, showing you exactly what it proposes to do. The section ends with what happens when you press Stop halfway through, which turns out to be one of the harder problems here and was, until recently, got wrong.

Agent mode turns a conversation into a plan-act loop. The model receives the tool schemas alongside the conversation; any tool calls it returns are executed, their

results appended as `tool` messages, and the loop repeats until the model stops calling tools or a step ceiling (fifteen) is reached. The step ceiling is a blunt instrument, but for a local tool it is the right kind of blunt: it guarantees termination without trying to be clever about what "stuck" means.

The ceiling alone turned out not to be enough, because it bounds the *length* of a run without protecting the steps inside it. Two narrower guards sit under it, and both exist because a single bad turn could otherwise consume the whole run. A model that emits the same tool call with byte-identical arguments repeatedly — a malformed argument blob, or a small model stuck in a groove — is cut off after three attempts and told so in a tool result it can act on, rather than being allowed to spend every remaining step, and every associated approval prompt, on the same failing call. And a step that returns neither text nor a tool call is retried twice with a short backoff instead of being treated as the model having finished; an empty response is a transport hiccup far more often than it is an answer, and reading it as completion silently truncated runs that had not finished. Each guard is deliberately shaped to end the *step* and continue the run, not to abort the run.

The builtin tools, and how each is gated, are the heart of the agent's safety posture:

Tool	Capability	Guard
`shell`	Run a PowerShell command	**Per-call human approval**
`file_read` / `file_list`	Read and list text files	Path sandbox
`file_write`	Write a text file	Free in workspace; **approval** outside it
`web_search`	DuckDuckGo search	—
`web_fetch`	Fetch a URL as readable text	**Per-call approval**; refuses private IPs
`memory_save` / `memory_delete` / `memory_search`	Durable facts	Ungated, fully user-visible
`knowledge_search`	Retrieve indexed passages	Ungated, read-only
MCP server tools	Whatever the registered server exposes	**Per-call approval**

The approval gate. The one-way NDJSON stream cannot pause for a dialog box, so consent arrives out of band. When the agent proposes a gated call, the loop emits an `approval_required` event carrying a fresh approval id, then parks on an `asyncio.Future` registered under that id. A separate endpoint, `POST`

`/api/approvals/{id}`, resolves the future when the user clicks Approve or Deny in the UI; the loop wakes and either runs the tool or reports "the user denied this tool call" as an ordinary tool result — so a denial redirects the model rather than crashing the run. Approvals have a timeout so an abandoned run cannot park forever, and they are single-shot: an unknown or already-used id is rejected. There is deliberately no "always allow."

The exit contract. A loop that can be cancelled at any await needs a defined answer for every way it can end, because the assistant message carrying a turn's tool calls is written to the database *before* any tool runs. Both the OpenAI and Anthropic wire formats reject an assistant tool call with no answering tool result, so a run that dies between those two writes leaves a conversation that fails validation on every later turn — permanently, since the rows are already on disk. The loop therefore closes its own gaps. The tool phase is wrapped so that every exit path — a finished step, a provider error, the step ceiling, a denial, or a cancellation from Stop or a dropped connection — records a result for each call it announced, handing that write to a task outside the cancelled scope because a cancelled context cannot await anything. Results are recorded *before* they are streamed to the client, so a disconnect at the moment of announcement cannot discard work a tool genuinely completed. Every termination also persists a short note saying which ending it was, so a transcript never simply stops without explanation.

Reconstruction applies the same rule defensively rather than trusting the loop to have been the only writer: assembling history synthesizes a placeholder for any tool call it finds unanswered. That repairs conversations damaged before the contract existed, and any that a power loss or a hard kill still manages to interrupt. The two layers are deliberate — the loop handles the graceful cases precisely, and the repair pass covers the cases where nothing graceful was possible.

Per-call, argument-aware gating. Gating is not a static property of a tool but a decision made per invocation, through a `needs_approval(args)` hook. This is what lets `file_write` be frictionless inside the agent's own workspace yet require approval the moment its resolved target lands in a user-granted external folder — the same tool, two risk profiles, distinguished by the actual arguments.

The file sandbox. Relative paths resolve inside a dedicated workspace folder. Absolute paths are honored only inside folders the user has explicitly granted under Settings; every other path is refused with an error string the model can read and adapt to. Grants are data, surfaced to the model in its system prompt so it knows where it may work, and revocable at any time. Crucially, paths are

re-resolved to their real location before the check, so a symlink or junction cannot be used to step outside a granted root.

Refusing the writes that destroy data. Containment answers *where* the agent may write; it says nothing about whether a permitted write is a good idea. Because the file tools offer whole-file writes while reads are truncated at a limit, the two compose into a data-loss shape: the model reads the first portion of a large file, edits what it saw, writes the result back, and the remainder is gone — an operation inside the sandbox, executed exactly as asked, that destroys content nobody ever looked at. Two checks refuse it. Every truncated tool result carries a shared marker, so content still containing that marker is provably a partial view and is rejected outright. And a truncated read is remembered along with the file's real size, so a later write-back *shorter* than the partial view it came from is refused with an explanation the model can act on. Both refusals are ordinary tool results rather than exceptions, so the model can re-read in full and retry. A third case was simpler and worse: missing content used to coerce to an empty string, so a malformed call truncated its target to zero bytes.

MCP. Users can register stdio MCP servers by command line. Their tools are namespaced by server (`filesystem_read_file`) and merged into the agent's toolset, indistinguishable to the model from builtins.

Two properties of that merge are load-bearing, and both were originally wrong. MCP tools are **gated by default**. A registered server runs with the user's privileges and may expose anything — a filesystem server's write, a shell wrapper, an API client — and nothing in a tool's schema tells the application which. Leaving them ungated made registering a server the one action that silently widened what the model could do without asking, which is precisely the promise every other risky tool here keeps; treating the whole class as approval-worthy is the only defensible default when the capability is unknowable in advance. Second, a name collision can no longer shadow a builtin. Merging used to be resolved last-wins by the dispatch map, so a server exposing a colliding name would quietly replace the real tool — taking its approval gate with it — while the specs, built from the raw list rather than the map, offered the model the same name twice. Builtins are now collected first and a colliding MCP tool is skipped with a logged warning, so registering a server can add capability but never redefine one.

One implementation subtlety earned its place in the code comments: each server connection is owned by a single dedicated asyncio task, because anyio cancel

scopes must be entered and exited by the same task; other tasks only use the already-initialized session object.

7. Persistent memory

In plain terms. Things worth carrying from one conversation to the next — that you prefer short answers, what project you are working on, a decision you made last week. The AI writes these down as short notes and reads them back at the start of later conversations, so you are not repeating yourself. You can see the entire list and delete anything from it at any time, and nothing is remembered from ordinary chat — only from agent mode, where you have already opted into the AI doing things.

An agent that forgets everything between conversations is a weaker assistant than the model's weights allow. Memory addresses this with deliberate modesty. The `memory_save` tool records a short, distilled fact; the newest memories, within a character budget, are injected into the agent's system prompt at the start of each agent-mode conversation, so preferences, project context, and decisions carry forward. `memory_search` reaches the older ones the budget omits, and `memory_delete` removes what has gone stale.

Memory is intentionally ungated, and the reasoning is worth stating because it runs opposite to the caution elsewhere. A saved memory has no effect outside the application; every save appears in the transcript as a tool card; and Settings → Agent memory is an always-available surface to review, add, and delete entries by hand. The combination — no external effect, full visibility, a standing kill switch — makes a per-save approval prompt pure friction. Plain, non-agent chat never receives memories at all, so the feature is scoped to the mode that opts into agency. Memories ride only on the request-local system prompt; they are never baked into a conversation's stored prompt, so deleting a memory truly forgets it everywhere.

8. Knowledge: local retrieval

In plain terms. You can point the program at a folder of your own documents — notes, reports, code, PDFs. It reads them once and builds an index, rather like the index at the back of a book, except organised by *meaning* rather than by exact wording, so a search for "holiday pay" can find a passage that only says "annual leave entitlement". When you later ask a question, it finds the handful of relevant

passages and shows just those to the AI. This is how you can ask about far more text than the AI could ever read in one go, and none of it is uploaded anywhere.

Retrieval lets the workspace answer from documents far larger than any context window, without those documents ever leaving the machine — the local-first promise applied to the user's own corpus. Files or folders (text, code, PDFs) are indexed under Settings → Knowledge. Each source is extracted to text, chunked with a paragraph-aware sliding window that overlaps adjacent chunks so a fact split across a boundary is still retrievable, embedded through a provider that implements `embed()`, and stored as normalized `float32` vectors directly in SQLite alongside the chunk text.

The search itself is deliberately unsophisticated: embed the query, then compute a cosine similarity against every stored chunk and return the top matches. At the few-thousand-chunk scale this application targets, a brute-force dot product in pure Python completes in milliseconds, which buys a real architectural simplification — no vector database to run or bundle, no numpy dependency, and no index to keep consistent. The retrieval quality ceiling is lower than a dedicated engine's, but for "chat with my folder of notes" it is comfortably sufficient, and the cost is a few dozen lines.

Three properties keep it safe and honest. Indexing is sandboxed to the same allowed roots as the file tools, re-resolving each walked file so a junction or symlink cannot pull outside-the-sandbox content into the index. Changing the configured embedding model clears the index, because vectors produced by different models are not comparable and silently mixing them would return plausible-looking nonsense. And what gets indexed is now *chosen* rather than merely encountered.

That last one is worth a paragraph, because the failure was quiet. Pointing the indexer at a project folder walked straight into `node_modules`, `.git`, and `.venv` — and because collection was path-alphabetical and capped, the budget was frequently exhausted on dependencies before reaching anything the user had written. The index looked full and returned nothing useful. Dependency and build directories are now pruned by name, and everything dropped — pruned folders, unreadable files, the overflow past the cap — is reported back rather than silently omitted, so a disappointing index explains itself instead of appearing complete.

Decoding was the same class of quiet damage. Both the indexer and the attachment endpoint used `decode("utf-8", errors="replace")`, which never fails and substitutes U+FFFD for every byte it cannot read. For an attachment that is merely

ugly; for the index it is permanent, because the mangled text is what gets embedded and the vectors carry the damage for as long as the index lives. Decoding now tries a short ordered list of encodings and takes the first clean result, with binary detection done on the raw bytes — a NUL near the start — because latin-1 will decode a JPEG perfectly happily and by the time it is a string the evidence is gone. Retrieval surfaces to the agent as the read-only `knowledge_search` tool and is reused, as the next section describes, by deep research.

9. Deep research

In plain terms. Ask a question and this searches the web, reads the pages it finds, and writes you a short report that says where each claim came from, so you can check it. The important design choice is that it follows a fixed set of steps — plan, search, read, write — rather than working out what to do as it goes. That makes it far more dependable with the smaller AI models people run at home, which are good at one well-defined task at a time and unreliable when asked to steer themselves through fifteen.

Deep Research answers a question by reading the web and the local index, then writing a cited report. The notable design choice is that it is a **deterministic pipeline, not an autonomous agent loop**: plan, then gather, then read, then synthesize, as fixed stages. The reason is reliability with the smaller local models this application often runs. Asking a 7-billion-parameter model to correctly drive a fifteen-step tool loop toward a good report is a gamble; asking it to perform four well-scoped single-shot tasks is not. One completion turns the question into a handful of search queries; each query runs through web search and the candidate URLs are deduplicated; the top sources are fetched to readable text (under the same private-address protection as the agent's `web_fetch`) and the local knowledge index is consulted; a final streamed completion writes a Markdown report that cites its numbered sources, followed by a Sources list.

Because the fetched pages are untrusted text fed into a prompt, the synthesizer is hardened against injection: each source body is wrapped in explicit fenced delimiters and the system prompt states that fenced content is data to be summarized, never instructions to obey; source titles are sanitized before they enter the report so a crafted result cannot smuggle a Markdown image beacon or link. The whole run persists as an ordinary conversation, which means history,

export, and the sidebar work with no new machinery — a recurring dividend of routing new features through the existing normalized model.

10. The writing editor

In plain terms. A place to write documents, with the AI on hand for editing rather than conversation. Select a sentence and ask for it to be shortened, expanded or tidied, and the suggestion appears beside your text for you to accept or reject. Nothing is changed without your say-so, and your documents save themselves as you type.

The editor is a local document workspace with AI editing folded in. Documents autosave to SQLite. A user selects text and applies a preset action (Improve, Shorten, Expand, Fix grammar), continues from the cursor, or issues a custom instruction; the model's suggestion streams into a panel to accept or reject. The edit endpoint, `POST /api/documents/edit`, is stateless — it returns only the replacement text and never mutates a document server-side. The client owns the decision to splice an accepted suggestion into the document at the captured selection, which keeps the server a pure text transformer and the document the single source of truth in the browser.

Three implementation details reflect hard-won correctness. Autosave is a dirty-flag debounce backed by a stale-proof mirror of the loaded document, and it *flushes* rather than merely cancels when the user switches documents or leaves the editor — so a fast edit-then-switch cannot silently drop the last change. The text area is locked while a suggestion is pending, because an accepted suggestion is spliced at offsets captured when the run started; allowing edits in between would let those offsets drift and land the replacement in the wrong place.

The third is subtler and is the reason this list grew. A textarea that has lost focus keeps reporting the selection it had when focus left, and the browser stops rendering it — so clicking Shorten after clicking away acted on a highlighted range the user could no longer see, and could not have known was still live. The caret is now trusted only while the textarea actually holds focus; otherwise the action falls back to the end of the document, and an action that genuinely requires a selection says it needs one rather than inventing a target. Selection-dependent controls also say which they are, so the distinction is visible before the click rather than discovered after it.

11. The blind arena

In plain terms. A way to find out which AI is genuinely better at the things *you* ask, rather than trusting a published benchmark. The same question goes to two of them, the two answers appear side by side with the names hidden, and you pick the one you prefer. Only then are the names revealed. Repeat a few times and you have a scoreboard built from your own judgement.

The arena exists to answer a question the multi-provider core makes cheap to ask: which model is actually better for *my* prompts? It runs one prompt against two chosen models with their identities hidden — a coin flip decides which model fills which on-screen column, so position never leaks identity. Both answers stream in parallel through two calls to `/api/complete`, a stateless single-shot completion endpoint that the arena shares with any future feature needing a one-off generation. The user votes — A, B, tie, or both bad — which reveals the names and records the outcome to a local per-model win/loss/tie leaderboard. Almost all of the feature is a view; the routing that makes it possible was already built for chat.

12. The model cookbook

In plain terms. AI models come in different sizes, and one too large for your computer will either crawl or refuse to start — which is a miserable way to discover the limit. This page looks at what hardware you actually have and says plainly which models will fit comfortably, which will be tight, and which will not run, then downloads the ones you choose. It is a convenience rather than a gatekeeper: models obtained any other way work exactly the same.

The cookbook is the one subsystem that reaches slightly outside the "model is a plugin" frame, and it does so carefully. It detects local hardware — CPU, RAM, and GPU VRAM, the last via `nvidia-smi` with a WMI fallback — using only the standard library and degrading every probe to "unknown" rather than raising, so a machine it does not fully understand still gets a usable page. It then rates a curated catalog of models as *fits your GPU* / *tight* / *CPU* / *too big* for that hardware, using rough per-model footprint estimates presented honestly as estimates. Models can be downloaded straight into Ollama with streamed pull progress and removed again.

The design keeps the plugin philosophy intact by being strictly *additive*: the cookbook never becomes the way models are served or selected. It only helps get weights onto disk via Ollama's native API, after which those models appear

through the ordinary provider and the normal model picker. Downloading is therefore Ollama-specific — other backends get recommendations but not one-click installs — which is an honest limitation rather than a hidden coupling.

The catalog is deliberately short, and that is a position rather than an oversight. A curated list is only refreshed when a release is cut, so the longer it grows the more of it is quietly out of date — a comprehensive-looking directory that is wrong in a dozen places is worse than a handful of suggested starting points that are right. It now holds roughly one entry per size band and says in as many words that it is not a model directory, which keeps the maintenance honest and points the user at the provider picker for everything else. Hardware detection, which shells out to external processes, runs off the event loop so a cookbook page load can never freeze a concurrent chat or research stream.

13. Interoperability

In plain terms. Other programs on your computer can borrow the AI setups you have configured here, instead of you entering the same accounts and keys into every tool separately. Your PC becomes the one place models and keys live, and anything that speaks the common industry format — a coding assistant, a script you wrote — can use them by pointing at a single address.

Syrudas is both a *client* of the OpenAI dialect, through `openai_compat`, and a *server* of it. The `/v1` surface exposes every model of every configured provider under a namespaced id (`ollama-local/llama3.1:8b`), and `chat/completions` translates each request into the normalized interior and routes it to the right backend, with streaming, tool calls, and usage preserved. These calls are stateless and never touch conversation history.

This inverts the usual integration burden. Rather than Syrudas writing an adapter for every editor and tool that might want to use it, any OpenAI-compatible client adopts Syrudas as its backend by changing one base URL — and in doing so gains uniform access to every model the user has configured, local and hosted alike, behind a single endpoint. The Continue extension for VS Code is the reference consumer, and gains something a chat window cannot offer — inline completions and edits applied directly in the editor. Command-line tools such as aider work the same way.

An earlier release also shipped a dedicated VS Code extension that embedded the workspace UI in an editor panel. It was removed: it wrapped the same web app in

an iframe rather than integrating with the editor, so it duplicated what a browser tab already did while adding a separate distribution channel and a version-drift surface. The one capability worth keeping — launching a chat prefilled from an external caller — was never the extension's to begin with. It is a `?prompt=` parameter honored by the web app itself, and remains available to any caller.

14. File attachments

In plain terms. You can drag a file onto a message. The program pulls the readable text out of it and includes that text in what it sends. It handles plain text, code, spreadsheet exports and PDFs. Anything it cannot read — an image, a scanned document with no text in it — it refuses with a clear explanation rather than quietly sending gibberish.

Attachments follow a stateless pipeline that mirrors the rest of the system's preference for keeping state in one place. The client uploads a file to `POST /api/attachments`; the server extracts text — UTF-8 for text-like formats, pypdf for PDFs, binaries rejected, with a character cap and explicit truncation flags — and returns it. The client then embeds that text into the outgoing message inside `<file name="...">` delimiters.

Storing attachment content *inside the message* rather than in a side table of blobs buys three properties at once: replaying history to any provider needs no special casing, because the file is just message text; backup and export are simply the database file; and the UI reconstructs collapsible per-file chips from the message text alone. The trade-off is larger message rows, which is acceptable at the capped sizes. For documents too large for the model's context window, the knowledge subsystem (Section 8) is the better path, and the two features compose: attach for a one-off, index for a corpus.

15. Security and privacy model

In plain terms. What this program can and cannot honestly promise about keeping your things private — including where it falls short, because a security section that lists only strengths is an advertisement. The short version: it accepts connections from your own machine and no other, it never reports anything to anyone, and the genuinely risky things the AI can do all stop to ask you first. The longer version below names the parts that remain imperfect, including one that was found and fixed while this edition was being written.

The threat model is honest about what a local, single-user tool can and cannot promise, and the posture is layered.

- **Network exposure.** The server binds to `127.0.0.1` exclusively, and a middleware rejects any request whose `Host` header is not a loopback name. Binding alone is not enough: without the Host check, a web page the user is merely visiting could POST to the local server via DNS rebinding and drive it from the outside. The two together close that vector. With no remote surface, there is deliberately no authentication layer to misconfigure.

- **Server-side request forgery.** The web-fetch path — used by both the agent and deep research — refuses any URL that resolves to a private, loopback, or link-local address, and re-checks on every redirect hop, so a fetched page cannot bounce the agent onto localhost, the application's own API, or the local network.

- **Model-initiated actions.** The genuinely dangerous capabilities — arbitrary shell, web fetch, file writes outside the workspace, and every MCP tool — are each gated per call with no persistent allow. File access is deny-by-default outside the workspace plus explicit grants, and writes that would destroy unread content are refused inside it (Section 6).

- **Instruction/data separation.** Both loops now draw the boundary explicitly. Deep research fences untrusted source text in begin/end markers, strips counterfeit markers a page may itself contain, and instructs the model to treat the contents as information even where the text claims otherwise. Agent mode applies the same treatment to *every* tool result, naming the tool that produced it, so a fetched page or a retrieved passage cannot present itself as something the user said. The fence is applied identically in two places that must not disagree — as results are appended during a live run, and as history is reassembled on replay — because a boundary that exists only in one of them is a boundary the next turn forgets. Fencing happens *before* trimming, so the budget accounts for the markers rather than being silently overrun by them. The previous edition of this paper listed agent-mode fencing as tracked work; it is done.

- **Static file serving.** The route that serves the single-page application resolves each request path against the bundled web assets and refuses anything that resolves outside them, falling back to the application shell. Resolution also collapses symlinks, so a link placed inside the bundle cannot point out of it. Containment matters more here than for an ordinary static-file route, because

these assets are served from the application's own origin: a local file served through this path would not merely be readable, it would execute — any HTML on disk would gain same-origin access to `/api` and `/v1`. That containment check arrived in version 1.0. Builds up to and including 0.7.4 resolved the path without it, so anyone still running one should update.

- **Data at rest.** Conversations, settings, memories, indexed chunks,

documents, and provider API keys live in one local SQLite file. Keys are stored in plaintext — an OS-keychain integration is future work — but are masked (`•••1234`) in every API response, so the browser never re-receives a full secret. The honest consequence, stated plainly in the setup guide, is that anyone with read access to the data folder can read the keys; the folder should be treated as being as sensitive as the keys themselves.

- **Telemetry.** None. The application makes outbound connections only to the

model backends the user configured and to URLs the user, or the user's agent under the rules above, requests.

- **Residual risks.** A granted folder is fully readable and writable by any

model the user runs, including a poorly aligned local one. MCP servers run with the user's privileges, so registering one is still an act of trust in the server itself — the per-call gate bounds what it can do unattended, not what it is. Tool arguments are not validated against the schema the tool declares before dispatch, so a malformed call is caught by the tool's own handling rather than at the boundary. And the unsigned executable requires a one-time SmartScreen click-through. These are documented rather than hidden, because a local tool's security story is only as good as its honesty about the edges.

16. Accessibility and theming

In plain terms. How the program looks, and whether it can be used comfortably by people who cannot use a mouse or who see colour differently. Two principles carry most of the weight. Colour never carries meaning on its own — anything shown in red or green also says so in words and with an icon — so nothing is lost if those colours read differently to you. And the light/dark setting is kept separate from the colour-vision setting, so you are never forced to accept a background you dislike in order to get a palette you can read.

Appearance and colour vision are treated as two independent axes, so a colour-blind user is never forced to choose between an accessible palette and a preferred background. Appearance (system, light, or dark) and colour vision (default, protanopia, deuteranopia, tritanopia, achromatopsia) are applied as attributes on the document root and resolved entirely through CSS variable overrides, with a pre-render inline script setting them before first paint to avoid a flash of the wrong theme. Colour-vision modes remap only the four status hues and are tuned per background so status *text* clears WCAG AA contrast on both light and dark; achromatopsia drops hue entirely and relies on luminance. Underpinning all of it is a rule the UI follows regardless of palette: status is never conveyed by colour alone — every state also carries an icon and a text label — so meaning survives any colour transformation, and the palettes are a legibility improvement rather than a load-bearing signal. The application also honours the operating system's reduced-motion preference.

Accessibility extends past colour. Every interactive control carries an accessible name — icon-only buttons included, each labelled with the action and its target (for example, the specific conversation a delete button removes) — and the conversation and document lists, which are visually rows, are exposed as keyboard-focusable buttons with the active row marked. Actions that appear on hover — the per-message copy and edit controls, the per-code-block copy button — are faded rather than hidden outright, because a control hidden with `visibility` is removed from the tab order entirely and can never receive the focus that would reveal it; kept at zero opacity they remain focusable and appear on keyboard focus.

The honest boundary of that work is keyboard operability, not screen-reader support, and the distinction matters enough to draw. Named controls and a correct tab order are necessary for a screen reader and not sufficient for one: the application currently declares no live region, so streamed replies and tool state changes are not announced as they arrive, and the collapsible tool card remains the one control driven by pointer events alone. A user navigating by keyboard is well served today; a user listening is not yet, and the gap is specific enough to close — a live region on the transcript, message-level status announcements rather than per-token ones, and keyboard semantics on that card. That work, and a screen-reader pass to validate it, is Section 19 material.

17. Testing and assurance

In plain terms. How the project checks its own work. Every part has automated tests that run the real program against a stand-in for the AI, so they finish in seconds and need no internet connection. More interesting than the list of what is tested is the question of when a test can be *trusted*: a test that would pass whether or not the thing it checks is broken is worse than no test, because it gets mistaken for safety. Two examples in this section show how that was checked rather than assumed.

A one-person codebase of this breadth stays trustworthy only with a testing discipline that matches. Every subsystem ships with an offline test suite that drives the *real* code paths against fakes rather than mocks of its own logic: a scripted fake provider exercises the agent and research loops through their actual event handling; `httpx.MockTransport` stands in for the Anthropic and Gemini wire protocols and for Ollama's native API; a deterministic fake embedder makes retrieval ranking meaningful without a model. Because the fakes sit only at the true boundaries — the network, the model — the suites verify the system's own behavior, and they need no network, no GPU, and no running model, so they run anywhere in seconds.

Cancellation is tested on the same principle, because it is the behavior least likely to be exercised by hand and the most expensive to get wrong. A dedicated suite drives the real loop, interrupts it mid-tool-call and mid-approval, and asserts what must hold afterwards: no tool call left unanswered, no approval id leaked, a recorded reason for the ending, a completed result never replaced by a placeholder, and a history that still assembles into a valid provider request. Fakes make those assertions cheap and deterministic; because a scripted provider cannot reproduce the fragmented, interleaved tool calls a real backend streams, the same properties were also confirmed against a live local model.

Choosing where to put the boundary is itself load-bearing, and the path containment of Section 15 is the clearest case. Its suite starts a real server on a loopback port rather than using an in-process test client, because such a client normalizes the request URL before the request is constructed — which silently repairs a percent-encoded traversal, the one form that survives the server's own normalization and reaches the handler intact. Tested through the convenient boundary, the suite would have passed over a live hole. Its assertions were then checked by removing the containment and confirming the suite fails, on the

principle that a security test that cannot fail is worse than no test at all, because it is mistaken for coverage.

Database migration is tested on the same principle, and for the same reason: it is the one code path whose failures are unrecoverable. A migration check that only ever inspected the `messages` table would have passed on a developer's machine and raised on every existing install the moment a column was added to any other table — the worst possible distribution of that bug. Migrations are now a versioned list stamped into `PRAGMA user_version`, each step written to be safe against a database that already has the change, and the file is copied aside before any of them run. A database created by a *newer* build is refused rather than read through a schema this code does not understand, because the alternative is writing over it.

That property is what makes the suites enforceable rather than merely available. A single entry point runs every offline suite alongside the frontend's unit tests, lint and typecheck, and continuous integration invokes that same entry point on every push and pull request, on Windows because that is the platform the application targets. The suites needing a live model backend sit behind a `-Smoke` switch on the same entry point rather than in a separate procedure, because a check that requires remembering a different command is a check that stops being run. The value is less in the automation than in the guarantee it removes from human memory: a regression in any subsystem fails the branch that introduced it, rather than waiting for whoever next thinks to run the suite by hand.

The client is covered on the same principle. Adversarial review kept finding its defects in one place — the logic that folds a stream of events into a thread and rebuilds that thread from storage — because those are stateful transforms whose mistakes are invisible until a specific sequence occurs. That logic is therefore kept as pure functions outside the component: a reducer that folds one event into the item list, and a rebuild that turns saved messages back into it. Both are exercised directly, with event scripts standing in for a model, including the sequences that previously produced wrong output — an agent step that calls a tool without speaking, where the turn's token counts belong to no visible message and must be discarded rather than written onto the previous answer. Interaction that genuinely depends on the DOM, such as the sidebar's rename committing on Enter and blur but discarding on Escape, is driven through the rendered component instead.

What every suite above has in common is that it checks *mechanics*: that a cancelled run leaves a valid transcript, that a path cannot escape, that a migration does not destroy a database. None of them notice if an edit to the agent's system

prompt makes the model shell out for every file read. That gap is covered separately by a behavioural evaluation which runs the real loop against a real model and scores what it did — which tools, in what order, how many steps, and whether the final answer contains what it should. Because models are not deterministic, a case asserts a *shape* rather than an exact transcript, and the result is a score to compare before and after a change rather than a pass/fail gate. It is kept out of the automatic discovery the other suites use, deliberately, because it costs real model time; it is run when the prompts or tool descriptions change, which is exactly when nothing else would have noticed.

Beyond the suites, substantial features were put through an adversarial review before shipping: independent passes over the diff for correctness, security, frontend contract, and test coverage, with each finding challenged by skeptics before it was accepted, and every confirmed finding fixed and re-verified. Several of the correctness properties described earlier — the autosave flush-on-switch, the editor's offset lock, the research injection hardening, the cookbook's off-loop detection — are fixes that this process surfaced. The point is not that the code is flawless but that its most load-bearing behaviors have been attacked on purpose.

18. Packaging and distribution

In plain terms. How the program reaches you. It is one folder you download, with one thing inside it to double-click and a second folder — `_internal` — holding the machinery that makes it run. The two have to stay together; everything else in there is yours. Anything the program saves — your conversations, your settings — goes in that same folder. Nothing is installed into Windows itself, so moving it somewhere else is dragging a folder, and removing it is deleting one. This section also explains why it is a folder rather than the single file it used to be, which turns out to be a story about antivirus software rather than about packaging.

The release artifact is a portable zip holding four things: the PyInstaller application (windowed, over WebView2) as an executable plus the `_internal` runtime folder beside it, an end-user README, the MIT license, and the optional provider connectors. It deliberately holds nothing else.

The choice of `--onedir` over `--onefile` is worth stating, because the single file is the more obvious answer and was the original one. A onefile build is a self-extracting archive: each launch unpacks the entire Python runtime into a temporary directory and executes from there. That costs start-up time on every run, and — more

importantly for something distributed to strangers — it is behaviourally indistinguishable from how a great deal of malware is packaged. Antivirus heuristics are tuned accordingly, and an unsigned self-extractor with no download reputation is the worst case: Microsoft's machine-learning classifiers flag exactly this shape, under names like `Wacatac.B!ml`, on files that are entirely benign. Onedir lays the runtime out as ordinary files that are read in place, so there is no extraction step to resemble. The cost is that the executable is no longer self-contained — it will not start without `_internal` beside it, which the end-user README says in as many words and the release verification checks before it launches anything. That cost is close to zero here, because the artifact was already a folder inside a zip; a user who was going to keep a folder together is not inconvenienced by it holding one more thing. None of this makes a false positive impossible. It removes the specific trait most likely to cause one, and the real fix — a code-signing certificate — remains unbought. Earlier editions also bundled this whitepaper and the setup guide, and both were wrong for the person receiving them: the setup guide was Markdown renamed `.txt`, so opening it produced a screenful of syntax, and half of it described building from source to somebody holding a finished executable. Both now live in the repository, linked from the README. The decision was not about size — they were 74 KB inside a 29 MB archive — but about what the folder looks like when the window opens, and how obvious the thing to double-click is.

Everything mutable lives beside the executable, so an installation is trivially copyable, movable, and deletable, and extracting a newer release over an older folder leaves the data untouched. On first run with an empty database, the server probes the well-known local backends — Ollama on `:11434`, LM Studio on `:1234` — and auto-configures any that respond, so a user who already runs a local model server reaches a working chat with zero configuration.

Version identity flows from `APP_VERSION` into the health endpoint, the UI footer, and — as of this revision — the cover of this document, substituted at render time rather than typed. One duplicate remains, and it is worth naming because the previous edition of this paper claimed the opposite: the Windows file properties of the executable are read from a separate `version_info.txt` that is maintained by hand, so cutting a release edits two files rather than one. Until that file is generated from `APP_VERSION` as well, the two can drift — and a shipped PDF whose cover disagreed with the build it accompanied is exactly what that drift looks like. The same version is now also stamped into the packaged README as the archive is

assembled, so a copy sitting in somebody's downloads folder can say what it is without being launched.

Three Windows-specific build lessons are preserved in the codebase for posterity. PyInstaller bundles are invisible to `pkgutil`, which is why builtin provider adapters are imported statically. PowerShell 5.1 under `$ErrorActionPreference = "Stop"` turns a native command's harmless stderr into a terminating error, which is why the build scripts wrap native calls in `cmd /c "... 2>&1"`. And a build script that hardcodes the path to a developer's own virtual environment runs nowhere else — not in a second checkout, not in continuous integration — which is why the packaging scripts now fall back to whatever `python` is on the path, as the test runner already did.

The finished archive is then unpacked somewhere clean and **actually launched**, and the release is refused if the packaged application does not come up and serve its interface. That check exists because version 0.7.3 shipped broken: everything had been verified from source and the archive's contents inspected, but the executable itself was never run.

That check then collided with another safeguard, in a way worth recording because the interaction is the general case rather than a one-off. A frozen build refuses to run from a temporary or extraction folder, because a user who double-clicks the exe straight out of the zip viewer gets a working application whose data lives somewhere Windows will later delete. Verification, however, unpacks to `%TEMP%` *on purpose* — so the guard fired on the release check and silently disabled the one test that catches a broken build. The resolution was an explicit opt-out the verification script sets for itself, rather than weakening the guard: a safety check that a process legitimately needs to bypass should be bypassed by name, in the open, by the one caller entitled to. The guard also compares fully resolved paths on both sides, because on a machine where the temp directory is reached through a symlink the unresolved comparison meant it never fired at all.

19. Limitations and roadmap

In plain terms. What this does not do well yet, written on purpose and in detail. Every project has a section like this; most leave it vague. First the fixed boundaries, then what the previous edition of this paper listed as broken and has since been fixed, then the things that still bite day to day, then the deeper

structural problems, then what is planned. If you only read one section of this paper to judge whether the project is honest, read this one.

The honest limitations are the mirror image of the design choices. This edition states the engineering ones alongside the product ones, because an earlier edition listed only the latter — and the engineering ones are what a reader evaluating the system would actually want to know. It also states, explicitly, which of them have been closed, because the failure mode of a section like this is not that it lies but that it stops being read against the code.

Product boundaries. The application is single-user by intent and Windows-only in a deeper sense than packaging: the shell tool spawns PowerShell directly, hardware detection reads WMI, the desktop shell targets WebView2, and the build and run scripts are PowerShell throughout. The web layer is portable Python, but a port would be genuine work rather than a packaging change, and everything below is written on the assumption that Windows remains the target. Multimodality is text-only: normalized message content is a string, a deliberate simplification that also means a vision-capable model can be selected but never fed an image. That mismatch was until recently advertised — the bundled catalogue recommended a vision model, and the attachment endpoint then refused images — and the catalogue entry has since been removed rather than left to promise something the program cannot accept. Key storage is plaintext on disk. Model *download* management is specific to Ollama.

Closed since the previous edition. Five items this section previously listed as open are now done, and are named here rather than quietly dropped, because a limitations section that only ever grows is as dishonest as one that is vague. The context budget is derived from the selected model's context window and re-applied every agent step rather than once per turn (Section 5). Agent mode fences tool output as data, in both the live loop and on replay (Section 15). MCP tools are gated per call, and can no longer shadow a builtin's name (Section 6). `file_write` refuses the partial-view write-backs that silently destroyed the unread remainder of a file (Section 6). And a run now has a record (below).

Runtime limitations. These matter more day to day than the boundaries above. The file tools offer read, whole-file write, and list — no patch-style edit, no pagination, no search — so changing part of a large file still means rewriting all of it. The data-loss shape that combination used to produce is now refused rather than executed, which converts a silent corruption into a visible inefficiency; it does not make the inefficiency go away, and a large file the model must edit in place

remains the case this toolset serves worst. Tool arguments are not validated against the schema the tool declares before dispatch, so a malformed call is caught by the tool's own handling rather than at the boundary, and the quality of that error varies by tool. The character-per-token ratio used to size the history budget is a heuristic standing in for a real tokenizer, so the budget is approximately right rather than exactly right — safe in the direction that matters, since it errs toward sending less, but not precise.

Instrumentation. A run now leaves a record as well as a transcript: a run identifier, per-step and per-tool timings, which calls were gated, which returned errors, the token total, and the reason the run ended — and a behavioural evaluation that scores agent behaviour against a real model when the prompts or tool descriptions change (Section 17). That closes the specific gap the previous edition named: the two highest-leverage knobs in the system can no longer be changed with no signal that behaviour moved. What is still missing is narrower but real. The record is a rotating log line rather than queryable storage, so answering "did this get slower over the last month" means reading logs, not running a query. There is no cost accounting in currency, only tokens. And the evaluation is a score compared by hand, not a threshold that fails a branch — which is the right shape while the case set is small, and the wrong one once it is trusted enough to gate on.

Structural. Agent mode and deep research are two orchestration loops that share a persistence helper and little else, so a fix applied to one is not a fix to the other. The tool interface accepts arguments but no run context, which is the pinch point for several items above. Both are the kind of change that grows more expensive with every feature layered on top, which is an argument for doing them earlier than their urgency alone would suggest.

Direction. In rough order of value: a patch-style file edit with pagination and search, which is the runtime limitation with the widest daily reach; schema validation of tool arguments at the dispatch boundary; run context threaded through the tool interface, which unblocks several of the above at once; image input carried through the same normalized schema; OS-keychain key storage; and an optional local token on the `/v1` hub for shared machines. Productivity integrations — email, calendar, notes — remain the largest gap relative to a full personal-assistant suite, and each is effectively its own subsystem that would be evaluated on its own terms rather than folded in for completeness.

A note on how this section is maintained, since it is the one most likely to rot. Everything above was checked against the code for this edition rather than carried

forward, which is how five items came to move into "closed" at once. A roadmap inherited unread is worse than no roadmap: it tells the reader the project does not know what it has built.

20. Conclusion

Syrudas AI is an argument, made in code, that a genuinely capable AI workspace — streaming chat, consent-gated agency, local retrieval, deep research, model comparison, a writing editor, protocol interoperability, and one-click distribution — fits inside a small, single-maintainer codebase when one abstraction is chosen well and enforced everywhere. Each feature in this paper is less an invention than a consequence: route it through the normalized model and the provider contract, and history, export, streaming, and provider-independence come for free. Models will keep changing, faster than anything else in the stack. A workspace whose only firm opinion about models is a five-method contract is built to outlast every one of them.

Put without the vocabulary: this is a program that lets you talk to an AI on your own computer, keeps everything you say on that computer, and can let the AI do real work for you while stopping to ask before anything risky. It is built so that the AI part can be replaced whenever a better one appears, without disturbing your conversations, your documents, or the way you work. AI models will keep changing quickly. The point of building it this way is that you should not have to.

Syrudas AI is open source under the MIT license. Architecture reference and setup: `README.md` *and* `docs/SETUP.md` *in the repository at github.com/LenSyrudas/syrudas.*